

Module 3: Naming

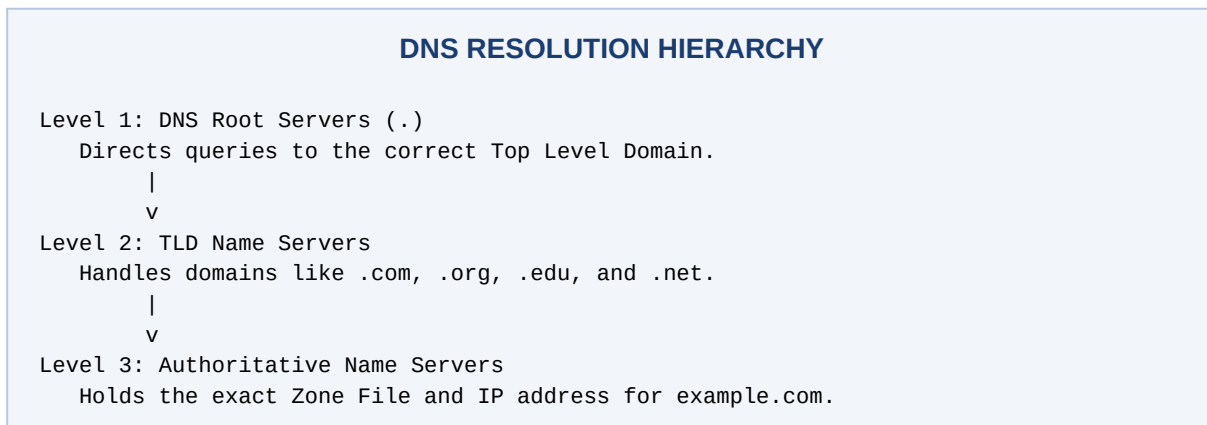
3.1 Generic Naming Schemes

In a distributed system, entities (files, servers, processes) must be named to be located and accessed. Not all names are created equal — the way a name is structured determines how easily it can be resolved to an actual location.

Naming Scheme	Description	Trade-off
Flat Naming	Raw identifiers with no structure, like a MAC address.	Unique, but gives no clue about location.
Structured Naming	Human-readable, hierarchical names (e.g., file paths, URLs).	Easy for humans; needs a resolution service.
Attribute-Based Naming	Locating resources by tags or properties (e.g., LDAP directories).	Flexible search, but can be slower to resolve.

3.2 Domain Name System (DNS)

DNS is the internet's premier structured naming system. It is decentralized and heavily hierarchical, converting human-readable domain names (like `www.example.com`) into IP addresses that computers actually use to route traffic.



In practice, most lookups do not walk the whole hierarchy every time. A **recursive resolver** (usually run by your ISP) does the multi-step walk on your behalf and then **caches** the answer for a period of time called the **TTL (Time To Live)**. This is why the very first visit to a new website can feel a touch slower than every visit after it.

■ The Student ID Generator Analogy

Think of naming and localization like a Student ID Generator system. A student's raw matrix number is like a flat name—it's unique but doesn't immediately tell you everything. The university portal (acting like DNS) takes that ID or the student's typed-in name, queries the backend directory (Authoritative Server), and resolves it to a specific database row containing their full profile, grades, and department.

3.3 Naming and Localization

Locating an entity based on its name requires specific techniques:

- **Broadcasting:** Shouting to the entire network to see who responds. Simple, but wasteful and does not scale to large networks.
- **Home-Based Approaches:** A "home" location always knows the entity's current whereabouts (similar in spirit to how Mobile IP tracks a moving phone).
- **Distributed Hash Tables (DHT):** A highly scalable peer-to-peer localization method where nodes form a mathematical ring (e.g., the Chord algorithm), mapping keys to specific nodes efficiently without any centralized server.

■ How a DHT Like Chord Works, in Plain Terms

Every node and every piece of data gets hashed to a number on a big circular number line. A piece of data is stored on the first node whose ID is equal to or comes right after the data's hashed value on that circle. To find data quickly without asking every single node, each node keeps a small "finger table" of shortcut pointers further around the ring, so a lookup takes only about $\log(n)$ hops instead of checking every node one by one.

■ Key Takeaway

Naming is really a two-part problem: choosing a scheme that is meaningful to humans (structured or attribute-based), and then efficiently resolving that name down to a flat, locatable address — which is exactly what DNS and DHTs do, just for very different scales and use cases.

Glossary of Key Terms

Term	Meaning
Flat name	An unstructured, unique identifier with no locational meaning.
TTL	Time To Live — how long a cached DNS answer stays valid.
Authoritative server	The DNS server that holds the definitive record for a domain.

Term	Meaning
DHT	Distributed Hash Table — a decentralized key-to-node mapping structure.
Chord	A specific DHT algorithm that arranges nodes in a hashed logical ring.

What You Should Know

- The difference between flat, structured, and attribute-based naming, and the trade-offs of each.
- The step-by-step hierarchical flow of a DNS query from the Root to the Authoritative server, including caching and TTL.
- How Distributed Hash Tables (like Chord) enable scalable peer-to-peer localization without a central directory.